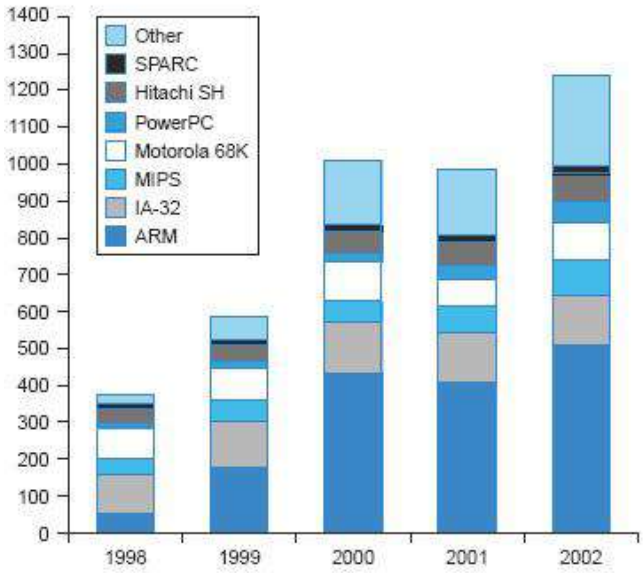
Linguagem de máquina

Instruções

- Linguagem da máquina (Assembly)
- Vamos trabalhar com a arquitetura do conjunto de instruções MIPS
- **M**icroprocessor without **i**nterlocked **p**ipeline **s**tages

  - Semelhante a outras arquiteturas desenvolvidas desde a década de 1980. Quase 100 milhões de processadores MIPS fabricados em 2002

  - Usada pela NEC, Nintendo, Cisco, Silicon Graphics, Sony...



Quando usar o Assembly

O motivo principal para programar em assembly, em vez de uma linguagem de alto nível disponível, é que a velocidade ou o tamanho de um programa possuem importância crítica. Por exemplo, imagine um computador que controla um mecanismo qualquer, como os freios de um carro. Um computador incorporado em outro dispositivo, como um carro, é chamado de *computador embutido*. Esse tipo de computador precisa responder rápida e previsivelmente aos eventos no mundo exterior. Como um compilador introduz incerteza sobre o custo de tempo das operações, os programadores podem achar difícil garantir que um programa em linguagem de alto nível responderá dentro de um intervalo de tempo definido – digamos, 1 milissegundo após um sensor detectar que um pneu está derrapando. Um programador assembly, por outro lado, possui mais controle sobre as instruções executadas. Além disso, em aplicações embutidas, reduzir o tamanho de um programa, de modo que caiba em menos chips de memória, reduz o custo do computador embutido.

CISC X RISC

CISC ("Complex Instruction Set Computer")

Conjunto de instruções complexo: grande número de instruções, muitos modos de endereçamento e tamanhos e formatos distintos.

RISC ("Reduced Instruction Set Computer")

Conjunto de instruções reduzido (simplicidade): algumas instruções simples com poucos formatos e modos de endereçamento (**implementação mais eficiente).**

CISC – Conjunto de instruções complexas

Características dos processadores CISC

- Muitas instruções.
- Instrução decodificada por firmware.
- Poucos registradores de propósito geral.
- Exemplos: Pentium, Athlon.

Pipeline

- Divide o caminho de dados do processador em estágios distintos.
- Uma instrução por estágio por ciclo de relógio.
- Aumenta o paralelismo e, portanto, o desempenho do processador.
- Originalmente desenvolvido para os processadores CISC.
- Primeiro pipeline: IBM 7030 (“Stretch”).

RISC – Conjunto de instruções reduzidas

RISC

- Poucas instruções.
- Complexidade no software.
- Decodificador de instruções em hardware (fixo).
- Todas as instruções têm um tamanho fixo (em geral, uma palavra de máquina).
- Todas as instruções requerem quase o mesmo tempo de execução.
- Vários registradores de propósito geral.

Ganho de desempenho do RISC em relação ao CISC

- Melhor uso dos pipelines.
- Rapidez na execução de instruções comuns.
- Menor quantidade de acesso à memória.

Pós-RISC

Processadores modernos

- Divergem dos projetos RISC e CISC tradicionais.
- Compreendem tudo o que pode melhorar o desempenho.
- Nomes comuns: pós-RISC, RISC de segunda geração e computação com conjunto de instruções rápidas (FISC).

Convergência do RISC para o CISC

- Arquitetura superescalar.
- Execução fora de ordem.
- Previsão de desvio.
- Suporte de ponto flutuante e de processamento de vetor no chip.
- Instruções adicionais usadas com pouca freqüência.

Categoria	Características dos processadores CISC	Características dos processadores RISC
Tamanho da instrução	Variável, tipicamente de 1 a 10 bytes.	Fixo, tipicamente 4 bytes.
Decodificação da instrução	Via microcódigo.	Em hardware.
Número de instruções em ISA	Muitas (tipicamente várias centenas), incluindo muitas instruções complexas.	Poucas (tipicamente menos de uma centena).
Número de instruções por programa	Poucas.	Muitas (geralmente cerca de 20% mais do que para CISC).
Número de registradores de propósito geral	Quase sempre poucos (por exemplo, 8 no processador Intel Pentium 4).[84]	Muitos (tipicamente, 32).
Complexidade	No hardware.	No compilador.
Habilidade para explorar paralelismo por meio de pipeline	Limitada.	Ampla.
Filosofia subjacente	Implementar o máximo possível de operações.	Tornar rápido o caso comum.
Exemplos	Pentium, Athlon.	MIPS, SPARC, G5.

Aritmética MIPS

- Todas as instruções possuem três operandos
- A ordem do operando é fixa (destino primeiro)

Exemplo:

Código C:	a = b + c
Código MIPS:	add a, b, c

"O número natural de operandos para uma operação como adição é três... Exigir que cada instrução tenha exatamente três operandos, nem mais nem menos, está de acordo com a filosofia de manter o hardware simples."

Aritmética MIPS

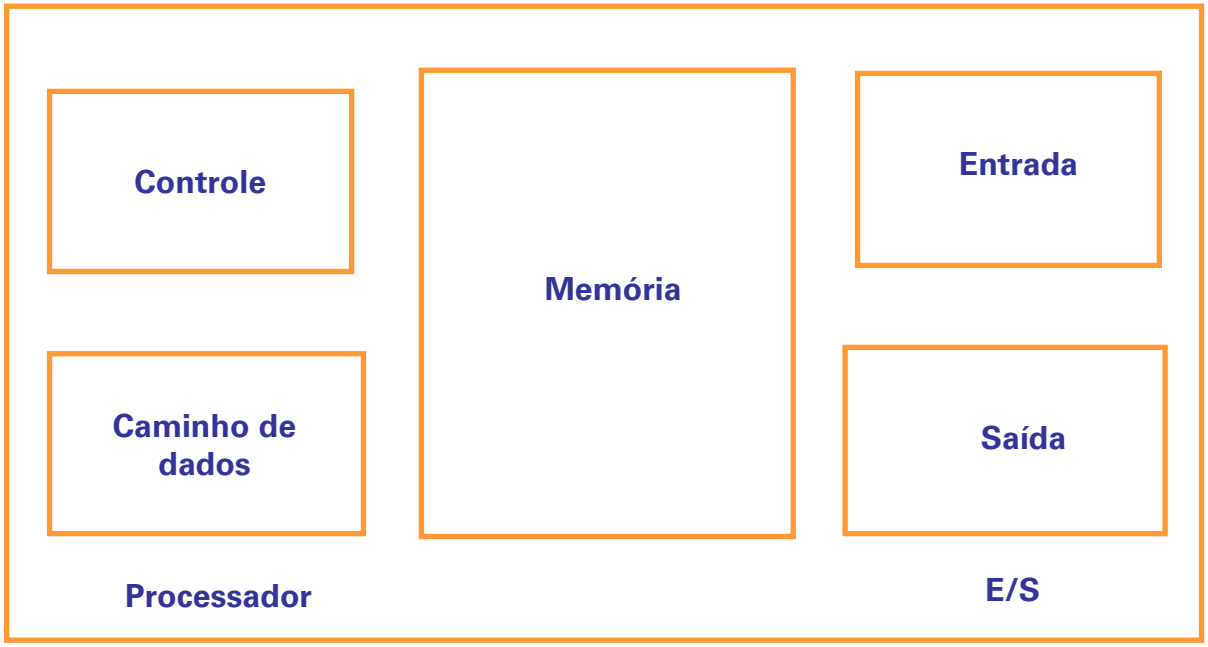
- **Princípio 1**: A simplicidade favorece a regularidade.

Código C:	a = b + c + d;
Código MIPS:	add a, b, c
	add a, a, d

- Os operandos precisam ser registradores, apenas 32 registradores fornecidos
- Cada registrador contém 32 bits
- Princípio de projeto: Quanto menor, melhor.

Registradores *versus* memória

- Os operandos das instruções aritméticas precisam ser registradores — apenas 32 registradores fornecidos
- O compilador associa variáveis com registradores



Registradores

- Benefício: Como registradores estão diretamente no hardware, eles são muito rápidos.
- Desvantagem: Como registradores estão em hardware, existe um número predeterminado deles.

- Solução: código MIPS deve ser muito cuidadosamente produzido para usar eficientemente os registradores.

- 32 registradores no MIPS

  Por que 32?

  Princípio 2: Menor é mais rápido (> no. reg → > ciclo clock)

  Cada registrador MIPS tem 32 bits de largura

  Grupos de 32 bits chamados uma palavra (word) no MIPS

Registradores do MIPS

Lista de associação de números e mnemônicos para os registradores do MIPS

Número (Decimal)	Nome
0	$zero
1	$at
2	$v0
3	$v1
4	$a0
5	$a1
6	$a2
7	$a3
8	$t0
9	$t1
10	$t2
11	$t3
12	$t4
13	$t5
14	$t6
15	$t7

Número (Decimal)	Nome
16	$s0
17	$s1
18	$s2
19	$s3
20	$s4
21	$s5
22	$s6
23	$s7
24	$t8
25	$t9
26	$k0
27	$k1
28	$gp
29	$sp
30	$fp
31	$ra

Convenção de uso dos registradores

Nome	Número do registrador	Uso
$zero	0	O valor constante 0
$v0-$v1	1-2	Valores para resultados e avaliação de expressões
$a0-$a3	3-5	Argumentos
$t0-$t7	6-15	Temporários
$s0-$s7	16-23	Valores salvos
$t8-$t9	24-25	Mais temporários
$gp	28	Ponteiro global
$sp	29	Ponteiro de pilha
$fp	30	Pointeiro de quadro
$ra	31	Endereço de retorno

Registrador 1 ($at) reservado para o assembler, 26-27 para o sistema operacional

Registradores

f = (g + h) – (i + j);

$s0, $s1,$s2,$s3,$s4 estão associados com f, g, h, i, j.

- Adição em Assembly (MIPS)

```
add $t0,$s1,$s2        # t0 = (g + h)
add $t1,$s3,$s4        # t1 = (i + j)
```

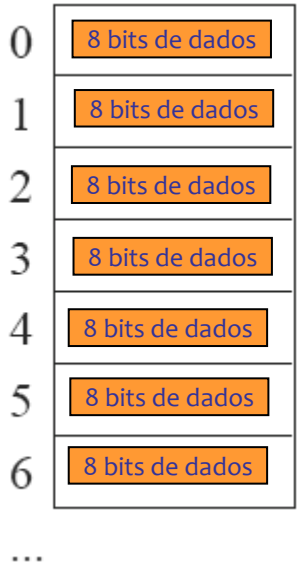
- Subtração em Assembly

```
sub $s0,$t0,$t1        # s0 = t0 - t1
```

Organização da memória

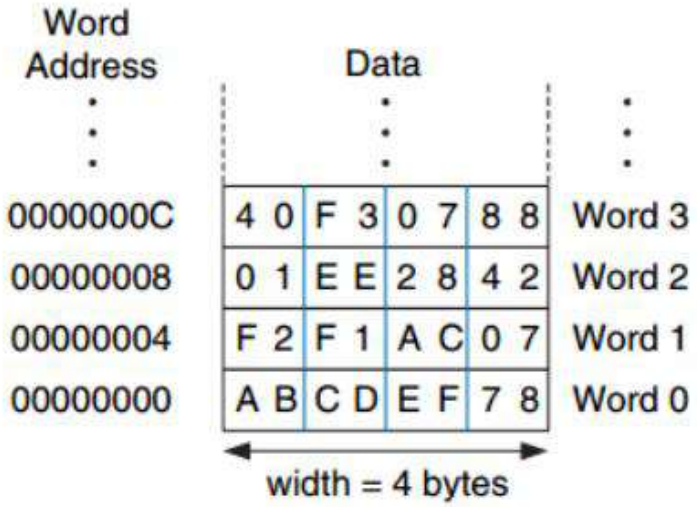
- Vista como um array grande e unidimensional, com um

  endereço.
- Um endereço de memória é um índice para o array.
- “Endereçamento de byte” significa que o índice aponta para um byte da memória.



Organização da memória

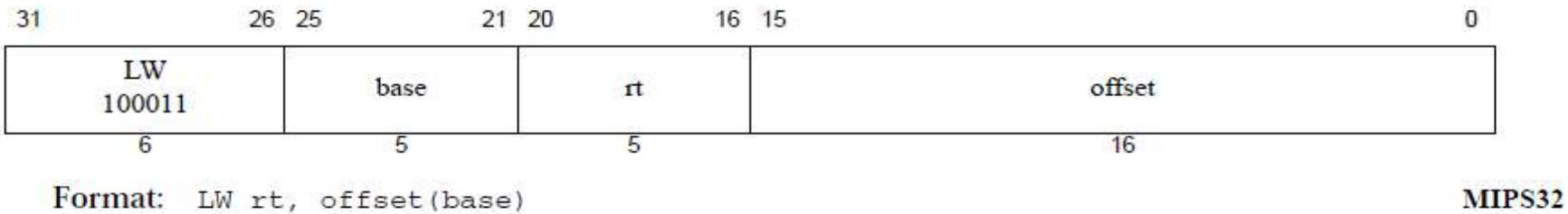
- Os bytes são bons, mas a maioria dos itens de dados usam “words” maiores
- Para o MIPS, uma word possui 32 bits ou 4 bytes.



2^{32} bytes com endereços de byte de 0 a 2^{32}-1
2^{30} words com endereços de byte 0, 4, 8, ... 2^{32}-4
As words são alinhadas

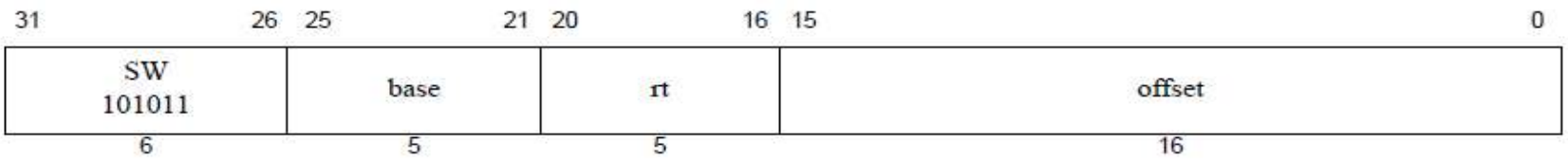
Instrução de carga

- LoadWord



- Lw $t0, 8($s3) # Registrador temporário $t0 recebe A[8]

Instrução de armazenamento



Format: `SW rt, offset(base)` MIPS32

sw $t0,12($s0)

Exemplo

A[12] = h + A[8];

g: $s1, h: $s2, $s3: endereço base de A

lw $t0,32($s3) # Registrador temporário *$t0 recebe A[8]*

add $s1,$s2,$t0 # Registrador temporário *$t0 recebe h+ A[8]*

sw $t0,42($s3) # Armazena h + A*[8] de volta em A[12]*

Instruções

- Instruções load e store
- Exemplo:

  Código C: A[12] = h + A[8];

  Código MIPS:
  ```
  lw $t0, 32($s3)
  add $t0, $s2, $t0
  sw $t0, 48($s3)
  ```
- Pode se referir aos registradores por nome (por exemplo, $s2, $t2) em vez do número
- A instrução store word tem o destino por último
- Lembre-se de que os operandos são registradores, não memória!

  Não podemos escrever: add 48($s3), $s2, 32($s3)

Até agora, aprendemos:

- MIPS

  - carga de words mas endereçamento de bytes

  - aritmética apenas em registradores

- **Instrução** **Significado**

Instrução	Significado
add $s1, $s2, $s3	$s1 = $s2 + $s3
sub $s1, $s2, $s3	$s1 = $s2 – $s3
lw $s1, 100($s2)	$s1 = Memory[$s2+100]
sw $s1, 100($s2)	Memory[$s2+100] = $s1

Linguagem de máquina

- Instruções, como registradores e words de dados, também possuem 32 bits de tamanho
  - Exemplo: add $t1, $s1, $s2
  - Registradores têm números, $t1=9, $s1=17, $s2=18
- Formato da instrução:

000000	10001	10010	01000	00000	100000
op	rs	rt	rd	shamt	funct

Linguagem de máquina

Pense nas instruções load-word e store-word

- Novo princípio: Um bom projeto exige comprometimento
  Introduza um novo tipo de formato de instrução
  - Tipo I para instruções de transferência de dados
  - Outro formato era o tipo R para o registrador

- Exemplo: lw $t0, 32($s2)

23	18	9	32
op	rs	rt	número de 16 bits

Instruções lógicas e de deslocamento

31–26	25–21	20–16	15–11	10–6	5–0
SPECIAL 000000	0 00000	rt	rd	sa	SLL 000000
6	5	5	5	5	6

Format: `SLL rd, rt, sa` **MIPS32**

31–26	25–22	21	20–16	15–11	10–6	5–0
SPECIAL 000000	0000	R 0	rt	rd	sa	SRL 000010
6	4	1	5	5	5	6

Format: `SRL rd, rt, sa` **MIPS32**

31–26	25–21	20–16	15–11	10–6	5–0
SPECIAL 000000	rs	rt	rd	0 00000	AND 100100
6	5	5	5	5	6

Format: `AND rd, rs, rt` **MIPS32**

31–26	25–21	20–16	15–11	10–6	5–0
SPECIAL 000000	rs	rt	rd	0 00000	OR 100101
6	5	5	5	5	6

Format: `OR rd, rs, rt` **MIPS32**

Constantes

- Constantes pequenas são usadas muito freqüentemente (50% dos operandos)

  Por exemplo: A = A + 5;
  B = B + 1;
  C = C - 18;

- Soluções

  - coloque "constantes típicas" na memória e carregue-as.

  - crie registradores "hard-wired" (como $zero) para constantes mais utilizadas.

- Instruções MIPS:

```
addi $29, $29, 4
slti $8, $18, 10
andi $29, $29, 6
ori $29, $29, 4
```

- Princípio de projeto: agilizar o caso comum.

Instrução LUI – Constantes de 32 bits

lui $t0, 255 # 16 bits superiores de $t0 recebem 255

Carregar esta constante em $s0:

0000 0000 0011 1101 0000 1001 0000 0000

lui $s0,61

31 – 26	25 – 21	20 – 16	15 – 0
LUI 001111	0 00000	rt	immediate
6	5	5	16

Format: `LUI rt, immediate` MIPS32

Purpose: Load Upper Immediate

To load a constant into the upper half of a word

Visão geral do MIPS

- instruções simples, todas com 32 bits
- muito estruturado, nenhuma bagagem desnecessária
- apenas três formatos de instrução

R	op	rs	rt	rd	shamt	funct
I	op	rs	rt	endereço de 16 bits		
J	op	endereço de 26 bits				

Controle

- Instruções de tomada de decisão
  - altera o fluxo de controle
  - por exemplo, mudar a "próxima" instrução a ser executada
- Instruções de desvio condicionais do MIPS:

```
bne $t0, $t1, Label
beq $t0, $t1, Label
```

- Exemplo:

```
if (i==j) h = i + j;

        bne $s0, $s1, Label
        add $s3, $s0, $s1
Label:....
```

Controle

- Instruções de desvio incondicionais do MIPS:

```
j  label
```

- Exemplo:

```
if (i!=j)                    beq $s4, $s5, Lab1
   h=i+j;                    add $s3, $s4, $s5
else                         j Lab2
    h=i-j;      Lab1:        sub $s3, $s4, $s5
                Lab2:  ...
```

Desvios

31 – 26	25 – 21	20 – 16	15 – 0
BEQ 000100	rs	rt	offset
6	5	5	16

Format: `BEQ rs, rt, offset` MIPS32

Purpose: Branch on Equal

31 – 26	25 – 21	20 – 16	15 – 0
BNE 000101	rs	rt	offset
6	5	5	16

Format: `BNE rs, rt, offset` MIPS32

Purpose: Branch on Not Equal

beq é branch if equal – Desvia se igual

bne é branch if not equal – Desvia se diferente

Desvios

- Além dos desvios condicionais, MIPS tem um desvio incondicional:

Format: `J target` MIPS32

Purpose: Jump

Chamada instrução Jump: pule (ou desvie) diretamente para o label

Decisões

Compile

```
if (i == j) f = g+h;
   else f = g-h;
```

.f até j = $s0 até $s4

Decisões

Código MIPS final compilado:

```
        bneq $s3,$s4, Else     # branch Else se i!=j
        add $s0,$s1,$s2        # f=g+h
        j Exit                 # vá para exit
else: sub $s0,$s1,$s2          # f=g-h
exit:
```

Laços

Laço tradicional em C

```
while (save[i] == k)
    i = i + 1;
```

```
Loop:   if (save[i] != k) goto Exit;
        i = i + 1;
        goto Loop;
Exit:
```

i até k = $s3 até $s5, base de save = $s6

Laços

```
Loop:   sll $t1,$s3,2          # $t1 = 4*i
        add $t1,$t1,$s6        # $t1=end(save+4*i)
        lw $t0,0($t1)          # $t1=save[i]
        bne $t0,$s5, Exit      # vai para Exit se save[i]!=k
        addi $s3,$s3, 1        # i=i+1
        j Loop                 # volta para Loop
Exit:
```

Testes

Desigualdade no MIPS

"Set on Less Than" – seta (poe 1) se menor que

31 – 26	25 – 21	20 – 16	15 – 11	10 – 6	5 – 0
SPECIAL 000000	rs	rt	rd	0 00000	SLT 101010
6	5	5	5	5	6

Format: `SLT rd, rs, rt` MIPS32

Purpose: Set on Less Than

se rs < rt, rd = 1;

caso contrário rd = 0;

SLTI

Seta se menor que (com uma constante)

Muito usada em laços for

31 – 26	25 – 21	20 – 16	15 – 0
SLTI 001010	rs	rt	immediate
6	5	5	16

Format: `SLTI rt, rs, immediate` MIPS32

Purpose: Set on Less Than Immediate

Função dos registradores

- Os registradores $a0-$a3 (4-7) são usados para passar os quatro primeiros argumentos às rotinas (os argumentos restantes são passados na pilha).

- Os registradores $v0 e $v1 (2, 3) são usados para retornar valores das funções.

- Os registradores $t0-$t9 (8-15, 24, 25) são **registradores salvos pelo caller**, que são usados para manter quantidades temporárias que não precisam ser preservadas entre as chamadas .

- Os registradores $s0-$s7 (16-23) são **registradores salvos pelo callee**, que mantêm valores de longa duração, que devem ser preservados entre as chamadas.

- A instrução jal escreve no registrador $ra (31), o endereço de retorno de uma chamada de procedimento.

JAL

31 – 26	25 – 0
JAL 000011	instr_index
6	26

Format: `JAL target` MIPS32

Purpose: Jump and Link

To execute a procedure call within the current 256MB-aligned region

Salva o endereço da próxima instrução em $ra e **pula para o label** dado

JR

31 – 26	25 – 21	20 – 11	10 – 6	5 – 0
SPECIAL 000000	rs	0 00 0000 0000	hint	JR 001000
6	5	10	5	6

Format: `JR rs` **MIPS32**

Purpose: Jump Register

To execute a branch to an instruction address in a register

Ao invés de prover um label para pular para, a instrução jr provê um **registrador** que **contém** um **endereço** para onde pular.

Formato das Instruções Tipo Registrador

R-type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Assembly Code	Field Values						Machine Code						
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct	
`add $s0, $s1, $s2`	0	17	18	16	0	32	000000	10001	10010	10000	00000	100000	(0x02328020)
`sub $t0, $t3, $t5`	0	11	13	8	0	34	000000	01011	01101	01000	00000	100010	(0x016D4022)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Formato das Instruções
Tipo Imediato

I-type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Assembly Code	Field Values				Machine Code				
	op	rs	rt	imm	op	rs	rt	imm	
`addi $s0, $s1, 5`	8	17	16	5	001000	10001	10000	0000 0000 0000 0101	(0x22300005)
`addi $t0, $s3, -12`	8	19	8	–12	001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
`lw $t2, 32($0)`	35	0	10	32	100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
`sw $s1, 4($t1)`	43	9	17	4	101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

Formato das Instruções
Tipo Jump

J-type

op	addr
6 bits	26 bits

	Machine Code								Field Values							Assembly Code
	op	rs	rt	imm					op	rs	rt	imm				
(0x2237FFF1)	001000	10001	10111	1111	1111	1111	0001		8	17	23	–15				addi $s7, $s1, -15
	2	2	3	7	F	F	F	1								
	op	rs	rt	rd	shamt	funct			op	rs	rt	rd	shamt	funct		
(0x02F34022)	000000	10111	10011	01000	00000	100010			0	23	19	8	0	34		sub $t0, $s7, $s3
	0	2	F	3	4	0	2	2								

Uso da memória

Em geral, pode ser necessário **salvar** algum outro **registrador** além de $ra.

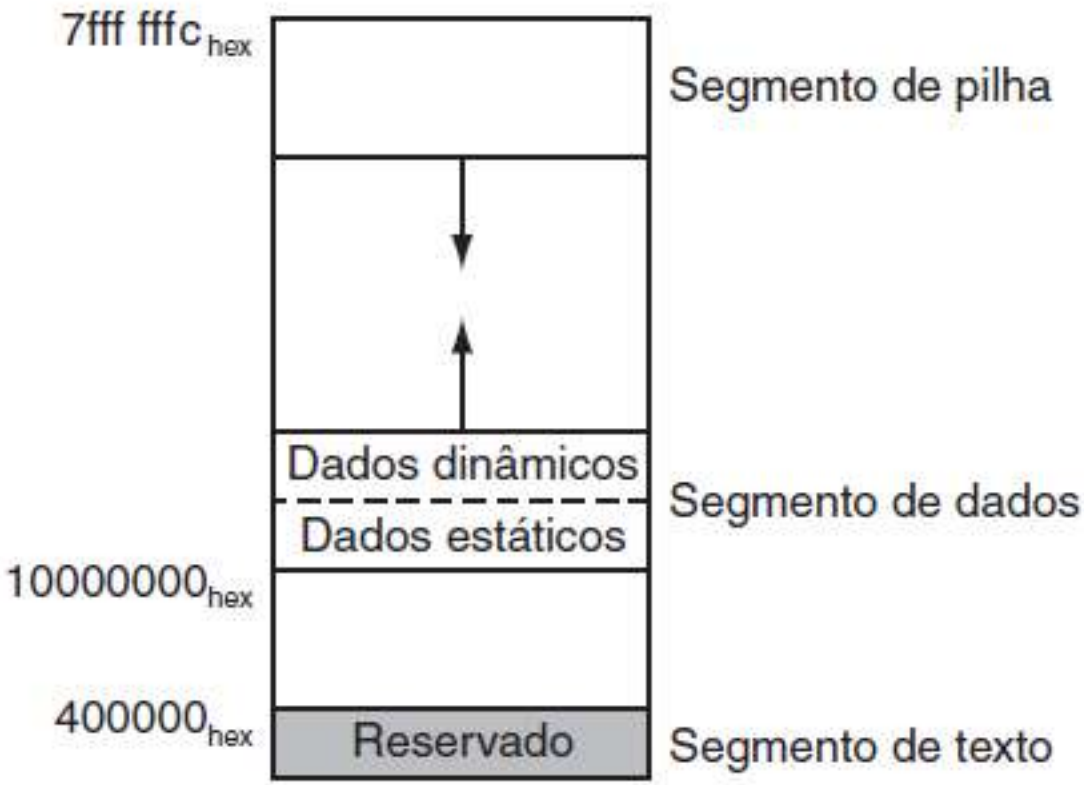
Quando um programa C está rodando, existem 3 **importantes áreas de memória** que são alocadas:

Static (alocação estática): Variáveis declaradas uma vez por programa, deixam de existir somente quando a execução termina.

Heap (alocação dinâmica): Variáveis declaradas dinamicamente.

Stack (pilha): Espaço a ser utilizado pela função durante sua execução; é aqui que podemos **salvar** os valores dos **registradores**.

Layout da memória



SP

Nós temos um registrador **$sp** que **sempre aponta** para o **último espaço utilizado na pilha**.

Para utilizar a pilha:

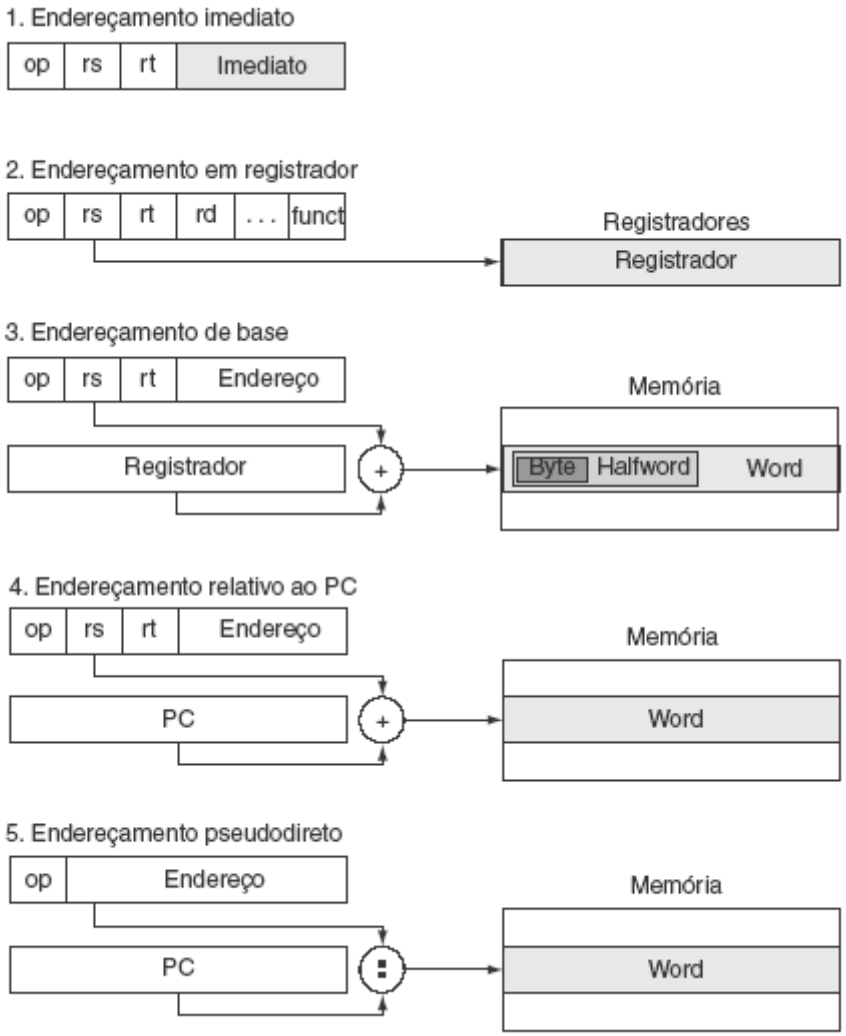
1 **decrementamos** o ponteiro **$sp** de 4 bytes

2 **preenchemos** os 4 bytes da pilha com a informação.

Então, como compilamos isto?

```
int sumSquare(int x, int y)
{
    return mult(x,x)+ y;
}
```

Modos de Endereçamento do MIPS



Modos de Endereçamento

Endereçamento Register-Only: O endereçamento register-only utiliza registradores para todos os operandos fonte e destino. Todas as instruções tipo-R utilizam endereçamento register-only.

Endereçamento Imediato: O endereçamento imediato utiliza os $16 - bits$ do imediato, juntamente com os registradores como operandos. Algumas instruções tipo-I, como adicionar imediato (addi) e carregar imediato superior (lui), utilizam endereçamento imediato.

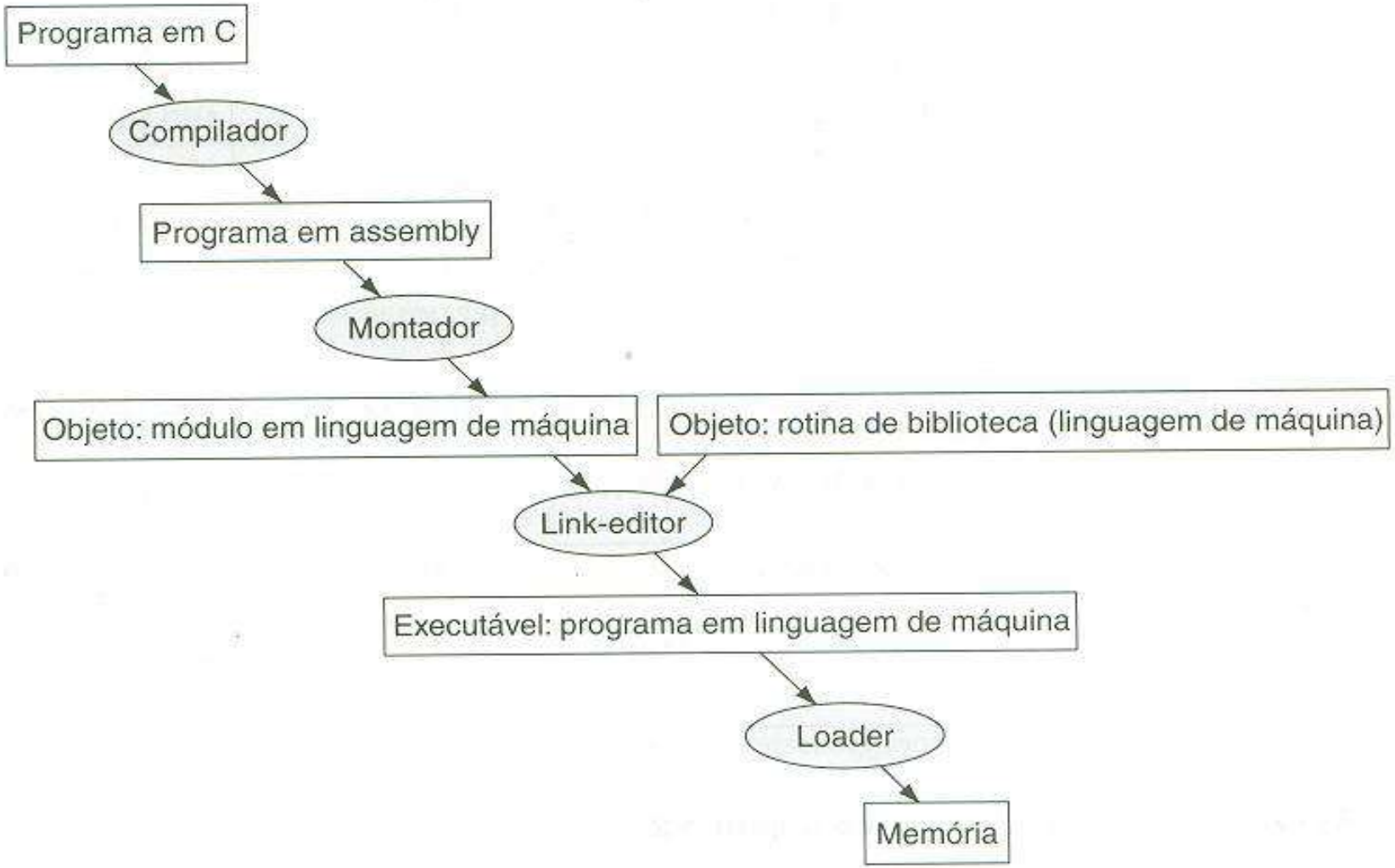
Endereçamento de Base: Instruções de acesso à memória, como carregar palavra (lw) e armazenar palavra (sw), utilizam endereçamento de base. O endereço efetivo do operando de memória é encontrado adicionando-se o endereço de base no registrador com o offset de $16 - bits$, estendido em sinal, encontrado no campo do imediato.

Modos de Endereçamento

Endereçamento Relativo ao PC: Instruções de branch condicional utilizam endereçamento relativo ao PC para especificar um novo valor para o PC se o desvio é realizado. O offset com sinal no campo do imediato é adicionado ao PC para obter um novo PC; consequentemente, o endereço de destino do branch é dito como sendo relativo ao PC atual.

Endereçamento Pseudo-direto: No endereçamento direto, um endereço é especificado na instrução. Os saltos de instruções, j e jal, deveriam, idealmente, utilizar endereçamento direto para especificar um endereço alvo de salto (jump target address – JTA) para indicar o endereço de instrução a ser executado em seguida.

Transformação C em Executável



Compilador e montador

Compilador

Transforma C Assembly.

Linguagem de alto nível: maior produtividade e menos linhas de código.

1975 – SOs escritos em escrito em Assembly.

Montador

O montador transforma um programa em Assembly em um arquivo objeto, que é uma combinação de instruções de linguagem de máquina, dados e informações necessárias para colocar intruções corretamente na memória. O montador cria as Pseudo-instruções que não existem na máquina melhorando para o programador os recursos do assembly.

Pseudo-instruções

move $t0,$t1 # $t0 ← $t1

O montador transforma em: **add** $t0,$zero,$t1

blt (branch if less than).

O montador substitui esta pseudo-instrução automaticamente por uma combinação de **slt/bne**.

bgt, bge, ble

Montador do MIPS: converte um **branch** fora da faixa em um **branch mais** um **jump**. Permite carregar **constantes de 32 bits** em um registrador, usando combinação de instruções.

O MIPS utiliza o registrador $at para uso exclusivo do montador.

Link-editor

- Um programa chamado link-editor "monta" os programas separados como um único programa determinando os endereços dos rótulos de dados e instruções

Assembly do MIPS				
Categoria	Instrução	Exemplo	Significado	Comentários
Aritmética	add	add $s1,$s2,$s3	$s1 = $s2 + $s3	Três operandos; dados nos registradores
	subtract	sub $s1,$s2,$s3	$s1 = $s2- $s3	Três operandos; dados nos registradores
	add immediate	addi $s1,$s2,100	$s1=$s2 + 100	Usada para somar constantes
Transferência de dados	load word	lw $s1,100($s2)	$s1 = Memória[$s2 + 100]	Dados da memória para o registrador
	store word	sw $s1,100($s2)	Memória[$s2 + 100] = $s1	Dados do registrador para a memória
	load byte	lb $s1,100($s2)	$s1 = Memória[$s2 + 100]	Byte da memória para registrador
	store byte	sb $s1,100($s2)	Memória[$s2+100] = $s1	Byte de um registrador para memória
	load upper immed.	lui $s1,100	$s1 = 100 * 216	Carrega constante nos 16 bits mais altos
Desvio condicional	branch on equal	beq $s1,$s2,25	if ($s1 == $s2) go to PC + 4 + 100	Testa igualdade; desvio relativo ao PC
	branch on not equal	bne $s1,$s2,25	if ($s1 != $s2) go to PC + 4 + 100	Testa desigualdade; relativo ao PC
	set on less than	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compara menor que; usado com beq, bne
	set less than immediate	Slti$s1,$s2,100	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compara menor que constante
Desvio incondicional	jump	j 2500	go to 10000	Desvia para endereço de destino
	jump register	jr $ra	go to $ra	Para switch e retorno de procedimento
	jump and link	jal 2500	$ra = PC + 4. go to 10000	Para chamada de procedimento